

1. String Memory & Address Fundamentals

To master string manipulation in C, you must understand exactly how the machine looks at a string inside your system's RAM.

The Single-Character Loop vs. Bulk %s

When you interact with arrays and strings, the **array name itself always evaluates to the base address** (the physical memory address of index 0).

Given: `char str[10] = "Bhopal";`
Base Address: 1000

```
str / &str[0] → [ 1000 ] → 'B'
                [ 1001 ] → 'h'
                [ 1002 ] → 'o'
                [ 1003 ] → 'p'
                [ 1004 ] → 'a'
                [ 1005 ] → 'l'
                [ 1006 ] → '\0' (Null Terminator)
```

Why `printf("%c", str[i]);` needs a loop:

`printf` with `%s` expects you to pass a **starting memory address pointer**. If you write `printf("%c", str[i]);`, you are passing a singular, raw text value (like 'B'), not an address—forcing you to use a loop to jump index-by-index manually.

Why `%s` handles it in bulk:

When you execute `printf("%s", str);`, you pass the base address 1000. The function reads the character at 1000, moves to 1001, and automatically prints continuously until it discovers the stop-flag `\0`.

Syntax Traps to Watch Out For

The Double-Address Error (&str): Writing `scanf("%s", &str);` or `printf("%s", &str);` is technically incorrect. The array name `str` *already* represents the base memory address. You do not need to append the address-of operator (`&`).

-
-

Sub-String Offsets: You can dynamically change your printing starting point by passing the address of a specific index element:

```
char str[30] = "Gaurav";  
printf("%s\n", &str[2]); ----> Passes the address of index 2 ('u')  
Output: urav
```

•

🔗 **Architectural Truth:** In C, the distinction between a character array and a string is purely based on **how the data is utilized by the programmer**. The compiler does not inherently know your array is a string; it relies completely on you to ensure it possesses a proper null terminator (`\0`).

📁 2. The Standard String Library (<string.h>)

To unlock pre-made string utility procedures, you must load the standard header file at the top of your program:

```
#include <string.h>
```

A. `strlen()` — String Length Calculation

Measures the number of active characters inside a string, **excluding** the null terminator (`\0`).

Technical Prototype :

```
size_t strlen(const char *string); ----> size_t is an unsigned integer alias
```

When a parameter expects a `char *`, it is explicitly asking for the **memory address** where the first character of your target string resides.

💻 Built-in Implementation

```
#include <stdio.h>  
#include <string.h>  
int main()  
{  
    char str[20] = "hello";
```

```
printf("Length of string = %ld\n", strlen(str)); ---> 'str' is passed directly because  
it acts as a pointer to the first  
element
```

```
    return 0;  
}
```

🌀 Behind-the-Scenes: Custom strlen (Without Library)

```
#include <stdio.h>  
int main()  
{  
    char str[30] = "Mahadev";  
    int count = 0;  
    int i = 0;  
  
    while (str[i] != '\0')  
    {  
        count++;  
        i++;  
    }  
    printf("Length of the string = %d\n", count); ----> Output: 7  
    return 0;  
}
```

B. strcat() & strncat() — String Concatenation

Appends a copy of the source string onto the tail end of your destination string.

⚠ Critical Buffer Overflow Warning

strcat() does **not** check if the destination array has enough room to fit the incoming text. If the combined length exceeds your array's buffer size, it will trigger severe **memory corruption and buffer overflow exploits**.

💻 Built-in Implementation

```
#include <stdio.h>  
#include <string.h>  
int main()  
{  
    char str1[30] = "Gaurav";  
    char str2[30] = "Kandpal";
```

```
strcat(str1, str2); ----> Appends str2 onto str1
printf("%s\n", str1); ----> Output: GauravKandpal
```

Pro Tip: String literals are stored as character arrays, meaning they automatically behave as pointers!

```
strcat(str2, "Gaurav");
printf("%s\n", str2); -----> Output: KandpalGaurav
return 0;
}
```

strncat() Sized Modification

To safely limit how many characters are appended, use `strncat()` by specifying a maximum character count:

```
char str1[30] = "Gaurav";
char str2[30] = "Kandpal";
strncat(str2, str1, 1); ----> Only appends the first 1 character of str1 ('G')
printf("%s\n", str2); ----> Output: KandpalG
```

Behind-the-Scenes: Custom strcat (Without Library)

```
#include <stdio.h>
#include <string.h>
int main()
{
    char str1[30] = "Gaurav";
    char str2[30] = "Kandpal";
    int i;

    int len1 = strlen(str1); ----> Find where str1 ends
    int len2 = strlen(str2);

    for (i = 0; i <= len2; i++)
    {
        str1[len1 + i] = str2[i]; ----> Copies characters, including the '\0'
    }
    printf("%s\n", str1); ----> Output: GauravKandpal
    return 0;
}
```

C. strcmp() — String Comparison

Compares two strings lexicographically based on their sequential **ASCII character values**.

Evaluation Rules

Returns **0** if both strings match identically.

Returns a **Negative Value (< 0)** if the first string is lexicographically smaller than the second.

Returns a **Positive Value (> 0)** if the first string is lexicographically larger than the second.

 **Common Interview Trap:** You *cannot* compare strings directly using raw print parameters like `printf("%d", strcmp(str1, str2))`; on some compiler platforms without saving the integer output first, as passing raw array labels compares their literal base address locations rather than the letters themselves!

Built-in Implementation

```
#include <stdio.h>
#include <string.h>
int main()
{
    char str1[30] = "Gaurav"; -----> 'G' ASCII value is 71
    char str2[30] = "gaurav"; -----> 'g' ASCII value is 103
    int result;

    result = strcmp(str1, str2); -----> 71 - 103 = Negative Value
    printf("%d\n", result); -----> Output: -1 (or any negative value)

    result = strcmp(str2, str1); -----> 103 - 71 = Positive Value
    printf("%d\n", result); -----> Output: 1 (or any positive value)
    return 0;
}
```

Behind-the-Scenes: Custom strcmp (Without Library)

```
#include <stdio.h>
int main()
{
    int flag = 0;
```

```

char s1[30] = "Hello";
char s2[30] = "Hillo";

for (int i = 0; s1[i] != '\0' || s2[i] != '\0'; i++)
{
    if (s1[i] != s2[i])
    {
        flag = 1;
        break; -----> Mismatch discovered; break out early
    }
}

if (flag == 1)
    printf("Strings are different\n");
else
    printf("Strings are same\n");

return 0;
}

```

D. strcpy() — String Copying

Completely overwrites the destination string buffer with the contents of the source string.

```

#include <stdio.h>
#include <string.h>
int main()
{
    char str1[30] = "Gaurav";
    char str2[30] = "Kandpal";

    strcpy(str1, str2); -----> Copies everything inside str2 into str1
    printf("%s\n", str1); -----> Output: Kandpal
    return 0;
}

```

Standard String Functions Quick Reference

Function Name	Purpose	Expected Arguments	Critical Safety Checks
---------------	---------	--------------------	------------------------

Function Name	Purpose	Expected Arguments	Critical Safety Checks
strlen(s)	Calculates length	(const char*)	Scans up until the first \0 byte marker.
strcat(d, s)	Appends string data	(char* dest, const char* src)	Dangerous: Destination must have extra size padding!
strncat(d, s, n)	Securely appends n bytes	(char* dest, const char* src, size_t n)	Safe alternative; stops buffer overflows.
strcmp(s1, s2)	Compares characters	(const char*, const char*)	Case-sensitive ASCII numeric math subtraction comparison.
strcpy(d, s)	Overwrites/Copies text	(char* dest, const char* src)	Overwrites baseline string data entirely up to \0.